

# Испит из Основних оптимизационих алгоритама у инжењерству

Напомене: Испит траје до 150 минута. На усмену одбрану испитног задатка долази се са попуњеним овим документом и одговарајућим прилозима (фајловима). Студент попуњава: своје податке у табели, датум испита, као и садржаје одељака од 1 до 5, уносећи одговарајуће податке уместо објашњења за попуњавање. Задатак се ради коришћењем рачунара, а коришћено развојно окружење или одговарајући софтверски пакет морају бити доступни за време испита. Студент може понети и свој рачунар. Задатак носи до 30 поена.

| Индекс<br>(година/број) | Презиме и име   | Датум       | Поени | Испит | Оцена |
|-------------------------|-----------------|-------------|-------|-------|-------|
| 2021/0379               | Виктор Живковић | 08.08.2025. |       |       |       |

## 1. Опис проблема оптимизације

Проблем који се решава је рачунање оптималних команди за прелазак игрице “Jump King” коришћењем генетичког алгорита. Идеја за овај проблем је потекла из јутјуб videa са канала “Code Bullet” [1], у коме он програмира генетички алгорита који игра игру и успева да победи.

“Jump King” је платформска видео-игра у којој играч управља ликом(Краљем) који се вертикално пење искључиво шетањем лево-десно и извођењем скокова чија се снага одређује дужином држања тастера. Недостатак контроле у ваздуху и могућност да и због најмање грешке играч изгуби велики део тока напретка чине ову игру изазовном.

Тренутни светски рекорд у брзини прелажења игре у категорији за људе је 3 минута, 59 секунди и 598 стотинки [2], а у категорији “TAS”(Tool-Assisted Speedrun) је 3 минута, 43 секунде и 973 стотинке [3].

Како је питање о низу уноса команди, које могу бити један од три тастера на тастатури, овај проблем се сврстава у групу SAT проблема.

Код оригиналне игре није доступан, тако да сам се ја одлучио за open-source рекреацију написану у програмском језику Python [4].

## 2. Дефиниција оптимизационе функције и оптимизационог простора

Навести да ли оптимизациони проблем има један или више критеријума. Написати формулу по којој се израчунава оптимизациона функција, за сваки критеријум. Дефинисати оптимизациони простор: навести оптимизационе променљиве, као и њихове границе.

Систем управљања сведен је на само три дугмета - лево за шетање лево, десно за шетање десно и горе за скок, јачина скока одређена је дужином држања тастера за скок.

Ради дефинисања оптимизационих променљивих и оптимизационог простора, ток играња једне “партије” је подељен на акције. Једна акција састоји се из команде (лево, десно, скок улево, скок удесно) и њеног трајања. Акција је представљена једним целим бројем од 8 бита, где прва 2 бита са највећом важности представљају код команде, а осталих 6 битова представљају дужину трајања акције, у “фрејмовима”(сликама).

Једна партија састоји се из низа акција, којих може бити произвољан број. Оптимизациони простор дефинише низ акција, посматраних једна за другом, као низ од  $8 * N$  битова, где је  $N$  број акција. Како димензионалност оптимизационог простора расте линеарно са бројем акција, а комплексност проблема експоненцијално зависи од димензионалности, партија је подељена на делове од 7 акција који се независно посматрају.

Ова одлука има смисла са обзиром да је циљ игре увек попети се што више за што краћи временски период. Међутим, постоји проблем са нивоима који захтевају привремено спуштање да би се ниво прешао, у оваквим нивоима постоји шанса да алгорита не нађе оптимално решење ако део који захтева спуштање не буде потпуно обухваћен једним оптимизационим блоком од 7 акција. Конкретно у овом раду алгорита још увек није стигао до таквог нивоа тако да је ово проблем који се може јавити у будућности развијања.

За рачунање оптимизационе функције неопходна је симулација која обухвата све акције из блока, из које само последња позиција играча и укупно време у игри улазе у прорачун оптимизационе функције.

Постоје 2 критеријума, висина коју је играч достигао и време које му је било потребно за то. Критеријуми се комбинују у једну оптимизациону функцију, тако да се критеријуму висине даје много већи значај. Један ниво је висине једног екрана, што је са нормалним подешавањима 720 пиксела. Време се мери у секундама и потом множи са 0,1 што значи да на крају игре ако је решење оптимално или близу оптималном биће отприлике 2,7 што је значајно мање од првог критеријума. Оптимизациона функција се рачуна на начин описан једначином 1, где је:

- $l$  тренутни ниво
- $h$  висина екрана у пикселима
- $y$  позиција играча на екрану у пикселима по вертикалној оси са почетком на врху екрана
- $t$  време.

$$f = (l + 1) * h - y - 0.1 * t \quad (1)$$

### 3. Оптимизациони алгоритам или алгоритми

За решавање проблема коришћен је генетички алгоритам представљен на предавањима [5]. Величина популације  $N_{pop}$  је 5000. Турнир селекцијом са величином турнира 1 селекује се  $\frac{N_{pop}}{5}$  родитеља.

Приликом укрштања, насумично се бирају 2 родитеља који се онда укрштају са вероватноћом 0,8 и у случају укрштања производе 4 деце. Имплементирано је бинарно укрштање са једном тачком пресека, за сваки пар родитеља укрштање се ради 2 пута, па пар родитеља производи 4 деце.

Вероватноћа мутације сваког бита у генерацији је 0,1.

За рачунање оптимизационе функције потребно је симулирање целе партије са потезима једне јединке, што је захтевно. Ради убрзавања овог процеса искључена је генерација визуелних слика приликом симулације играња и имплементирана је паралелизација која омогућава покретање задатог броја симулација у исто време. Овај број зависи од рачунара на коме се покреће оптимизација, за мој кућни рачунар оптималан број конкурентних симулација је 6.

Почетно решење генерисано је потпуно случајно.

### 4. Резултати

Приказати добијене резултате графички, табеларно или на други начин који је примерен задатку.

### 5. Прилози

Уз урађени задатак обавезно се прилаже (1) код програма који извршава оптимизацију, или (2) пројекат коришћен за оптимизацију у одговарајућем софтверском пакету. Навести имена приложених фајлова (или пројеката), као и кратак опис садржаја приложених фајлова, у једној до две реченице. Уколико постоји више прилога, пожељно је да буду запаковани у један zip-фајл. Овај попуњени документ, заједно са прилозима, послати на адресу [olcan@etf.rs](mailto:olcan@etf.rs) до почетка испитивања.

Код се налази на Github линку <https://github.com/Sslowpoke/Jumpkingathome> и у фолдеру `JumpKingAtHome/`.

Једини фајлови који су мењани у односу на оригинални код игре су `Brain.py` и `JumpKing.py`. У фајлу `Brain.py` налази се имплементација оптимизационог алгоритма, док се у фајлу `JumpKing.py` налази модификован интерфејс са остатком игре.

У случају покретања фајла `Brain.py` врши се оптимизација, а у случају покретања `JumpKing.py` могуће је играти игру нормално.

Свака етапа оптимизације чува резултате у фајлу чије је име време завршавања оптимизације а налази се у фолдеру `Saves/`, док се финално решење састојано из надовезаних резултата етапа оптимизације налази у фајлу `Saves/latest.txt`.

## Литература

- [1] Code Bullet, AI Learns to play JUMP KING, YouTube. <https://youtu.be/DmQ4Dqxs0HI>
- [2] Speedrun.com <https://www.speedrun.com/jumpking>
- [3] <https://www.youtube.com/watch?v=LZ1o6TzL7N4>
- [4] senweim, JumpKingAtHome, <https://github.com/senweim/JumpKingAtHome>
- [5] Драган Олћан, Предавања из предмета ООА, 2025.